# Solving the N-body Problem: Leapfrog and Runge-Kutta Integration

Jane Cohen[1]

[1] *Numerical Methods I, Master in Computational Physics, Université Marie et Louis Pasteur, Besançon, France*

(Dated: October 23, 2026)

# 1. Introduction

## 1.1. The N-Body Problem

The N-body problem involves predicting the motion of a system of $N$ interacting bodies. Classically, it refers to the motion of celestial bodies under their mutual gravitational interactions [1]. Given the masses, initial positions, and velocities of the bodies, the objective is to determine the evolution of the system over time. For a system of two bodies interacting through Newtonian gravity, an analytical solution can be obtained. However, when the number of bodies increases to three or more, the equations of motion become increasingly complex. This is because the motion of each body is influenced by the gravitational forces exerted by the other $N-1$ bodies, resulting in a system of coupled, nonlinear differential equations. In general, these equations cannot be solved analytically in closed form [2]. Therefore, numerical integration methods are required to approximate the positions and velocities of the bodies at later times.

## 1.2. Objectives of the Project

The objective of this project is to implement a numerical simulation of the N-body problem in Fortran. The bodies will be subject to the Newtonian gravitational force of all other particles in the system. Gravitational softening and other numerical techniques are employed to improve the numerical stability of the simulations. The behaviour of both two-dimensional (2D) and three-dimensional (3D) systems are investigated. Three numerical integration schemes are implemented and compared: leapfrog, fourth-order Runge–Kutta (RK4), and eighth-order Runge–Kutta (RK8). The conservation properties of each method are evaluated by examining the evolution of total mechanical energy, angular momentum, and barycentric velocity. Additionally, the computational complexity of the N-body problem is investigated, and the computational performance of the different integration methods are compared.

# 2. Physical Model and Initial Conditions

## 2.1. Newtonian Gravitational Interactions

In the proposed system, there are $N$ particles whose motion is described using Newtonian mechanics. The system has a total mass $M$ made up of particles of identical mass $m$. As the particles are assumed to be in a vacuum with no external forces acting on them, they interact purely through gravitational force. For a particle i, its motion is governed by the equation

$$m\frac{d^2\boldsymbol{X}_i}{dt^2} = \sum_{j\neq i} \frac{Gm^2}{|\boldsymbol{X}_j - \boldsymbol{X}_i|^3}(\boldsymbol{X}_j - \boldsymbol{X}_i) \tag{2.1}$$

where $\boldsymbol{X_i}$ is the position of particle i and G is the universal gravitational constant.

## 2.2. Dimensionless Units

To avoid numbers too large to be stored, dimensionless units are used. Taking $\tilde{X}_i = X_i/R$ and $\tilde{t} = t/t_0$, and substituting in $m = M/N$ the resulting equation of motion is

$$\frac{d^2\tilde{\boldsymbol{X}}_i}{d\tilde{t}^2} = \sum_{j\neq i} \frac{(\tilde{\boldsymbol{X}}_j - \tilde{\boldsymbol{X}}_i)}{N|\tilde{\boldsymbol{X}}_j - \tilde{\boldsymbol{X}}_i|^3} \times \frac{GMt_0^2}{R^3} \tag{2.2}$$

Defining $t_0$ such that the fraction $\frac{GMt_0^2}{R^3}$ is equal to one, the expression is simplified into

$$\frac{d^2\boldsymbol{X}_i}{dt^2} = \sum_{j\neq i} \frac{\boldsymbol{X}_{ij}}{N|\boldsymbol{X}_{ij}|^3} \tag{2.3}$$

where $\boldsymbol{X}_j - \boldsymbol{X}_i = \boldsymbol{X}_{ij}$ and the tildes of the dimensionless variables have been dropped to simplify the notation.

Here, the characteristic length $R$ and mass $M$ are given by $R = 50kpc$ and $M = 10^{11}M_\odot$, resulting in a characteristic timescale $t_0 = \sqrt{R^3/(GM)}$ (approximately 0.53 billion years). This system with these given characteristic values is designed to study the evolution of a galaxy.

### 2.3. Initial Spatial Distribution

Using the dimensionless units for $X_i$, the initial spatial distribution of the system is a random distibution inside a 2D circle or 3D sphere of dimensionless radius $R = 1$. Using a random seed, coordinates $[x, y]$ or $[x, y, z]$ are generated and rejected until they satisfy the condition of being inside the circle or sphere. This process is carried out to determine the initial position of all N particles.

### 2.4. Initial Velocity Distribution

The initial velocity of each particle is set according to solid body rotation. Here, each particle will rotate about the z-axis along a radius given by $r = \sqrt{x^2 + y^2}$ in both 2D and 3D. In the 2D case, all particles lie in the $xy$-plane, while in 3D, particles may have nonzero $z$-coordinates but still rotate about the same axis. The velocity of each particle at $t = 0$ is proportional to the radius $r$ and is given by $\boldsymbol{v}_i = \boldsymbol{\Omega} \times \boldsymbol{r}_i$ where $\boldsymbol{\Omega}$ is the angular velocity vector. Because the z-axis is defined as the axis of rotation, the only non-zero component of angular velocity is $\Omega_z$, giving the following equation for the components of the initial velocity of each particle i

$$\boldsymbol{v}_i = \boldsymbol{\Omega} \times \boldsymbol{r}_i = \begin{pmatrix} 0 \\ 0 \\ \Omega \end{pmatrix} \times \begin{pmatrix} x_i \\ y_i \\ z_i \end{pmatrix} = \begin{pmatrix} -\Omega y \\ \Omega x \\ 0 \end{pmatrix} \tag{2.4}$$

### 2.5. Barycentric Reference Frame

In its current state, the system will drift over time as the system is not in its barycentric frame. To avoid this drift and ensure the numbers stay within reasonable values, the system is changed to the barycentric frame. This is done by subtracting the motion of the barycentric center of the system from each velocity component for each particle. The barycentric velocity can be found using the following equation [3]:

$$\boldsymbol{v}_{bar} = \frac{m}{M}(\boldsymbol{v}_1 + \boldsymbol{v}_2 + ... + \boldsymbol{v}_i) = \frac{1}{N}\sum_{i=1}^{N} \boldsymbol{v}_i \tag{2.5}$$

## 3. Numerical Integration

### 3.1. Leapfrog Integration Scheme

To compute the state of the system at each time step, the leapfrog integration method was used. This method takes advantage of the fact that particle acceleration depends only on position, as shown in Equation 2.3. In this project, the Kick–Drift–Kick (KDK) formulation was implemented, a variation of the conventional staggered leapfrog scheme that is mathematically equivalent to the velocity Verlet method [4]. This formulation allows particle positions and velocities to be calculated and stored at the same time steps. The integration consists of three steps. First, a half-kick updates the velocity using the acceleration at the current positions:

$$\boldsymbol{v}_{n+1/2} = \boldsymbol{v}_n + \frac{\Delta t}{2}\boldsymbol{a}(\boldsymbol{X}_n). \tag{3.1}$$

Next, a drift advances the positions using the intermediate velocities:

$$\boldsymbol{X}_{n+1} = \boldsymbol{X}_n + \Delta t\, \boldsymbol{v}_{n+1/2}. \tag{3.2}$$

Finally, a second half-kick completes the velocity update using the acceleration at the new positions:

$$\boldsymbol{v}_{n+1} = \boldsymbol{v}_{n+1/2} + \frac{\Delta t}{2}\boldsymbol{a}(\boldsymbol{X}_{n+1}). \tag{3.3}$$

A visualization of these operations is shown in Figure 1.

### 3.2. Validation Using a Two-Body System

The integrator was first tested using two bodies in 2D circular orbits. The simulated trajectories and orbital period were compared to the analytical to determine the accuracy of the integration scheme. In a two body system, to have circular orbits, each particle must begin at the position across from the other, with velocities perpendicular to the circular orbit. The analytical trajectories of the particles are given by

$$\boldsymbol{X_i} = Rcos(\theta)\hat{x} + Rsin(\theta)\hat{y} \tag{3.4}$$

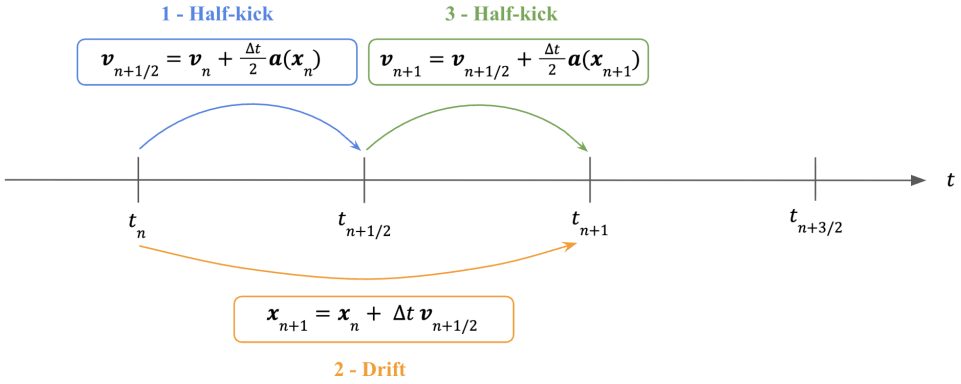



FIG. 1. Schematic representation of the Kick–Drift–Kick (KDK) leapfrog integration scheme. The first half-kick updates the velocity using the acceleration at $t_n$, followed by a full drift that advances the position to $t_{n+1}$. The second half-kick completes the velocity update using the acceleration at the new position.

As seen in the corresponding simulation in Figure 2, the numerical trajectories follow that of the analytical solution. The analytical solution predicts that the period will be

$$T = 4\pi\sqrt{\frac{R^3}{Gm}} \tag{3.5}$$

where R is the distance to each particle from the center of rotation and m is the mass of a particle. Using dimensionless units defined previously and an arbitrary position for the particles, $R = 0.5$, $m = \frac{1}{N} = 0.5$ and $G = 1$, resulting in a predicted period of 6.2832. The numerical solution from the leapfrog simulation also showed the particles completing one full rotation after 6283 steps, which with the time step used of 0.001, is a numerical period of 6.283. The test was repeated with different values of R and time steps, and the results between the analytical and numerical solutions agreed. Therefore, for this simple test of two bodies in circular orbits, the leapfrog integration scheme is able to predict the motion of the particles accurately.

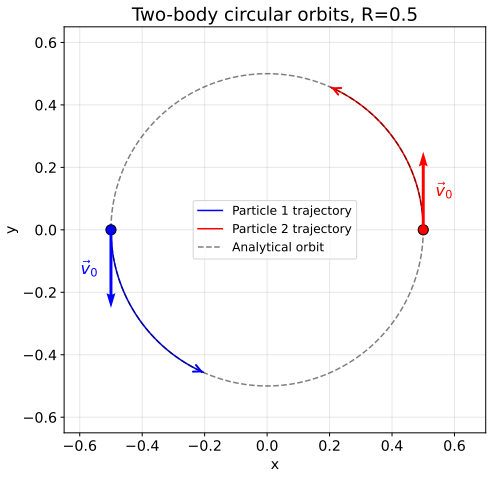



FIG. 2. Simulated trajectories of two equal-mass particles in circular orbits of radius $R = 0.5$. The dashed line represents the analytical orbit, the solid lines show the numerical trajectories, and the arrows indicate the initial velocities $\boldsymbol{v}_0$.

### 3.3. Evolution of a Many-Body System

After confirming that the two-body simulation behaved as expected, the system was expanded to $N = 1000$ particles to investigate the dynamics of a many-body gravitational system. The motion of the particles was observed by plotting the system at various time steps. Examples of these plots are presented in (Appendix D). The N-body system was then ready to be analyzed in detail.

### 3.4. Gravitational Softening

In an $N$-body system, the gravitational force increases rapidly as the separation between particles decreases. During close encounters, this can produce large accelerations that are difficult to resolve numerically. To regularize the gravitational force, a softening parameter $\epsilon$ is introduced, modifying the force law to [5]:

$$\boldsymbol{F}_i = \sum_{j \neq i} \frac{\boldsymbol{X}_{ij}}{N^2 \left(X_{ij}^2 + \epsilon^2\right)^{3/2}}, \tag{3.6}$$

where $\boldsymbol{X}_{ij} = \boldsymbol{X}_j - \boldsymbol{X}_i$ is the displacement from particle $i$ to particle $j$, and $X_{ij} = |\boldsymbol{X}_{ij}|$.
The effect of the softening parameter can be seen by simulating the gravitational acceleration during an idealized close encounter for various values of $\epsilon$. Without softening ($\epsilon = 0$), the acceleration increases sharply as the particles approach one another. Increasing $\epsilon$ reduces the maximum acceleration and produces a more constant force throughout the encounter.
To investigate the influence of gravitational softening on the dynamics of the full system, simulations were performed using $\epsilon = 0.001$, 0.01, 0.1, and 1.0, with identical time steps. By using the same random seed, identical initial conditions were generated for each simulation, allowing the resulting motions to be compared directly. Any differences in the predicted motion could therefore be attributed to the choice of time $\epsilon$.
By observing the trajectories of individual particles, it was apparent that varying $\epsilon$ produces significant differences in particle motion. However, the relationship between the trajectory of an individual particle and $\epsilon$ is not immediately obvious.
The influence of gravitational softening on the radial expansion of the system was examined by measuring the fraction of particles outside the initial boundary $R = 1$. It was shown that $\epsilon = 0.001$ produces the greatest radial expansion, while $\epsilon = 0.1$ results in substantially fewer particles crossing the boundary. Interestingly, $\epsilon = 1.0$ also produces significant expansion, demonstrating again that the relationship between softening and particle displacement is not linear.
These results illustrate the compromise involved in selecting the gravitational softening parameter. Very small values preserve strong short-range gravitational interactions but can produce large accelerations during close encounters, requiring smaller integration time steps to maintain numerical accuracy. On the other hand, excessive softening suppresses these interactions and modifies the dynamics of the system.
A value of $\epsilon = 0.01$ was therefore selected as a reasonable compromise for further investigation. Relative to the initial system radius ($R = 1$), this introduces softening over a comparatively small length scale while avoiding the dramatic effects of close interactions. However, additional convergence tests are necessary to determine whether this choice provides sufficient numerical accuracy.

### 3.5. Influence of the Time Step

To investigate the influence of the time step on the numerical solution, simulations were performed using $\Delta t = 0.1$, 0.01, and 0.001, with a gravitational softening parameter $\epsilon = 0.01$. A fixed random seed was again used to reproduce identical initial conditions.
Again, the trajectory of single particles and system radial expansion were observed to try and determine the effect of $\Delta t$ on the system and choose a suitable value for subsequent simulations. The trajectory of a single particle for all three time steps is the same at the start, but then they diverge significantly. In particular, $\Delta t = 0.1$ produces a substantially different trajectory from the smaller time steps. Although $\Delta t = 0.01$ and 0.001 show some agreement during the initial evolution, their trajectories also diverge as time progresses. This indicates that the numerical solution is sensitive to the integration time step.
In terms of the radial expansion of the system, $\Delta t$ was shown to have apparent effects on the dynamics of the system. For the largest time step, $\Delta t = 0.1$, the system undergoes rapid radial expansion, with nearly all particles moving beyond the initial boundary in both 2D and 3D within a relatively short period. In contrast, $\Delta t = 0.01$ and 0.001 predict considerably less expansion. These results suggest that $\Delta t = 0.1$ is too large to accurately resolve the gravitational dynamics for $\epsilon = 0.01$. However, to determine whether $\Delta t = 0.01$ or 0.001 are suitable, further investigation is required.

## 4. Conservation of Physical Quantities

### 4.1. Conservation of Mechanical Energy

For $N$ particles of equal dimensionless mass, the conservation of mechanical energy provides a measure of the numerical accuracy of the integration scheme and can help determine suitable values for $\epsilon$ and $\Delta t$. For each analysis, only the results for the 3D simulation are presented. In all cases, the 2D simulation produced results that agreed with the 3D simulation and the corresponding plots are in Appendix D.
The total kinetic energy is obtained by summing the kinetic energies of all particles:

$$E_{\text{kin}}(t) = \frac{1}{2} \sum_{i=1}^{N} \frac{1}{N} |\boldsymbol{v}_i(t)|^2. \tag{4.1}$$

Because the velocities and positions of each particle were saved at the same time step during the leapfrog integration, no adjustment to either value is required to calculate the kinetic energy.
The gravitational potential energy is obtained by summing over all distinct particle pairs. The factor 1/2 prevents double counting, while $\epsilon$ accounts for gravitational softening:

$$E_{\text{pot}}(t) = -\frac{1}{2}\sum_{i=1}^{N}\sum_{j\neq i}\frac{1}{N^2\sqrt{X_{ij}^2(t)+\epsilon^2}}. \tag{4.2}$$

The total mechanical energy $E_{tot}$ is the sum of the kinetic and gravitational potential energies:

$$E_{\text{tot}}(t) = E_{\text{kin}}(t) + E_{\text{pot}}(t). \tag{4.3}$$

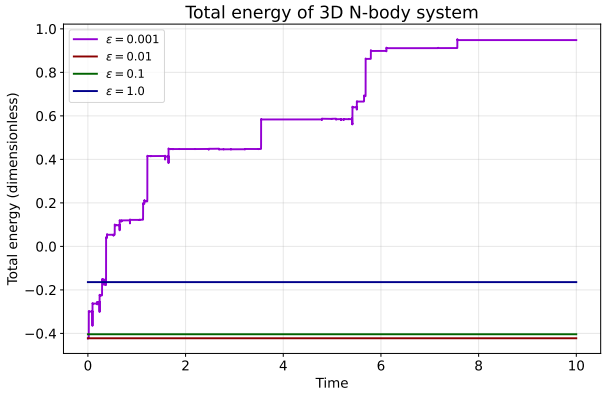
For a conservative system, the total mechanical energy should remain constant. Its relative variation is evaluated as

$$\frac{\Delta E(t)}{|E(0)|} = \frac{E_{\text{tot}}(t) - E_{\text{tot}}(0)}{|E_{\text{tot}}(0)|}. \tag{4.4}$$

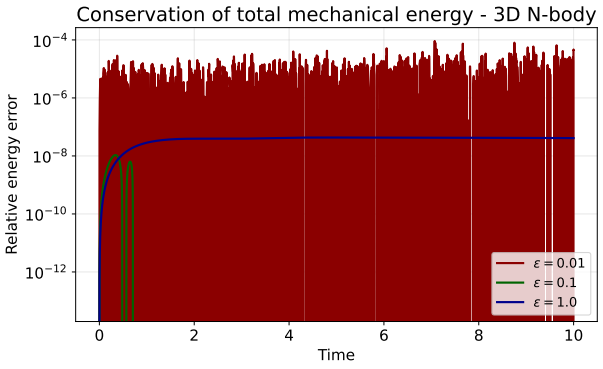
The total mechanical energy was conserved for every simulation that was run and analyzed. For example, for a simulation with a time step of $\Delta t = 0.01$ and $N = 1000$ particles, the total mechanical energy remained constant and the relative deviation of the total energy had a maximum value of 0.000507 (approximately 0.051%).
Using conservation of total energy as an indication of an accurate integration scheme, various parameters can be varied to determine their effect of the dynamics of the system.

### 4.1.1. Gravitational Softening

The influence of the gravitational softening parameter, $\epsilon$, was further investigated by examining the conservation of total mechanical energy. Simulations were performed using $\epsilon = 0.001$, 0.01, 0.1, and 1.0, with a fixed time step of $\Delta t = 0.001$.
Figure 3 compares the evolution of total energy and its relative deviation for these four values of $\epsilon$. For $\epsilon = 0.001$, the total mechanical energy increases dramatically in both dimensions, indicating significant numerical error. The energy increases occur in abrupt steps, consistent with poorly resolved close encounters producing large accelerations. This demonstrates that $\epsilon = 0.001$ is insufficient to maintain accurate energy conservation with the chosen time step.
In contrast, the simulations using $\epsilon = 0.01$, 0.1, and 1.0 maintain approximately constant total energy. The relative energy deviation reveals differences that are not visible in the total energy plots. For $\epsilon = 0.01$, the error reaches approximately $10^{-4}$, while $\epsilon = 0.1$ and 1.0 produce substantially smaller deviations, generally of order $10^{-8}$.




(a) Total mechanical energy




(b) Absolute relative energy deviation

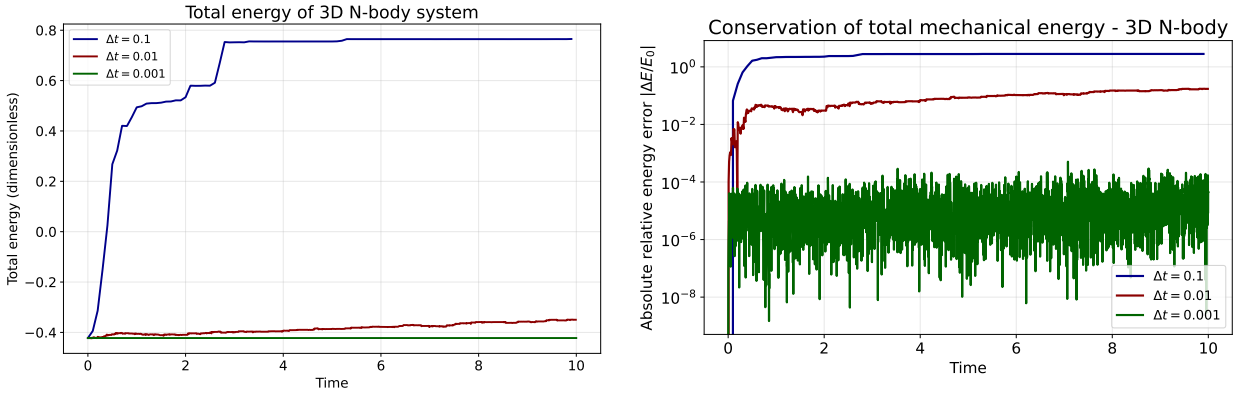
FIG. 3. Comparison of total mechanical energy conservation for the 3D $N = 100$ particle system over $t = 10$, using a time step of $\Delta t = 0.001$ and different gravitational softening parameters. Panel (a) shows the evolution of total mechanical energy for $\epsilon = 0.001$, 0.01, 0.1, and 1.0. Panel (b) shows the corresponding absolute relative energy deviation for $\epsilon = 0.01$, 0.1, and 1.0 on a logarithmic scale. The $\epsilon = 0.001$ case is excluded from panel (b) due to its substantially larger energy error.

Although larger softening parameters provide better energy conservation, they also result in substantial suppression of gravitational forces. Therefore, $\epsilon = 0.01$ was selected as a reasonable compromise between weakening gravitational interactions for close-encounters and maintaining numerical accuracy.

### 4.1.2. Time step

The influence of the integration time step on energy conservation was investigated using $\Delta t = 0.1$, 0.01, and 0.001, with a fixed gravitational softening parameter of $\epsilon = 0.01$.

Figure 4 shows the total mechanical energy and its absolute relative deviation. For $\Delta t = 0.1$, the total energy increases dramatically, indicating substantial numerical error. Reducing the time step to $\Delta t = 0.01$ improves energy conservation, although noticeable drift remains. In contrast, $\Delta t = 0.001$ produces small fluctuations, with relative energy errors generally below $10^{-4}$.
These results demonstrate the importance of sufficiently small time steps for resolving close gravitational encounters. Therefore, $\Delta t = 0.001$ was selected for subsequent simulations.




(a) Total mechanical energy

(b) Absolute relative energy deviation

FIG. 4. Energy conservation in the 3D $N = 100$ particle system for $\Delta t = 0.1$, 0.01, and 0.001, with gravitational softening $\epsilon = 0.01$. Panel (a) shows the total mechanical energy, while panel (b) shows its absolute relative deviation on a logarithmic scale.

### 4.2. Conservation of Angular Momentum

The total angular momentum is calculated by summing the angular momenta of all particles [6]:

$$\boldsymbol{L}(t) = \sum_{i=1}^{N} \frac{1}{N} \boldsymbol{X}_i(t) \times \boldsymbol{v}_i(t). \tag{4.5}$$

Since gravitational forces are central, the total angular momentum should remain constant. The magnitude of the components of the angular momentum and its relative deviation were analyzed for the three time steps. All three components remained constant throughout the simulation, with relative errors of order $10^{-15}$. This demonstrates that angular momentum is conserved to approximately machine precision, independently of the integration time step. Unlike energy conservation, angular momentum conservation therefore does not provide a useful criterion for selecting $\Delta t$ in this simulation. However, angular momentum being conserved helps confirm that the force calculations are being implemented correctly.

### 4.3. Conservation of Barycentric Velocity

The barycentric velocity is directly proportional to the linear momentum of the system as [7]:

$$\mathbf{P} = M\mathbf{v}_{\text{bar}} = \left( \sum_{i=1}^{N} m_i \right) \mathbf{v}_{\text{bar}}. \tag{4.6}$$

Because the barycentric velocity was subtracted from the initial particles, the system began with zero linear momentum. As there are no external forces acting on the system, there should be no addition of linear momentum during the system evolution. The barycentric velocity during a simulation was analyzed for the three time steps. In all cases, the barycentric velocity remains close to zero, with deviations of order $10^{-15}$. The negligible change in barycentric velocity confirms that the system conserves total linear momentum regardless of the time step.

## 5. Computational Performance

### 5.1. Theoretical Computational Complexity

The direct $N$-body method calculates the gravitational acceleration of each particle by summing the contributions from the remaining $N - 1$ particles. Since each pairwise interaction requires a fixed number of arithmetic operations, evaluating the accelerations of all particles requires

$$N_{\text{interactions}} = N(N - 1). \tag{5.1}$$

In the original Kick–Drift–Kick leapfrog implementation, the accelerations were calculated twice per time step, resulting in $2N(N-1)$ pairwise interactions. However, the acceleration calculated during the second half kick is identical to that required for the first half kick of the following iteration. To eliminate this repeated calculation, the accelerations of all particles are stored and reused between time steps.
With this optimization, the accelerations are evaluated once before integration begins and once during each time step. For $K$ integration steps, the total number of pairwise force evaluations is therefore

$$N_{\text{interactions}} = (K+1)N(N-1), \tag{5.2}$$

compared with $2KN(N-1)$ in the original implementation.
The computational complexity is therefore quadratic in the number of particles:

$$T(N,K) = \mathcal{O}(KN^2). \tag{5.3}$$

For a fixed number of integration steps, this reduces to $\mathcal{O}(N^2)$.

### 5.2. Computational Time Measurements

To verify the theoretical computational complexity of the leapfrog integration scheme, simulations were performed in two and three dimensions for increasing numbers of particles, from $N = 10$ to $N = 5000$. The number of integration steps was fixed at $K = 1000$, and execution times were measured using the Fortran `cpu_time()` subroutine. The results are presented in Table I.

TABLE I. Execution times of the optimized leapfrog integration scheme in two and three dimensions for 1000 integration steps.

| $N$ | 2D Time (s) | 3D Time (s) |
|---|---|---|
| 10 | 0.0406 | 0.0533 |
| 100 | 0.7051 | 0.8213 |
| 500 | 14.52 | 16.13 |
| 1000 | 57.07 | 65.14 |
| 2000 | 226.80 | 250.70 |
| 5000 | 1460.90 | 1528.96 |

The measured execution times were first fitted using a second-degree polynomial,

$$T(N) = aN^2 + bN + c, \tag{5.4}$$

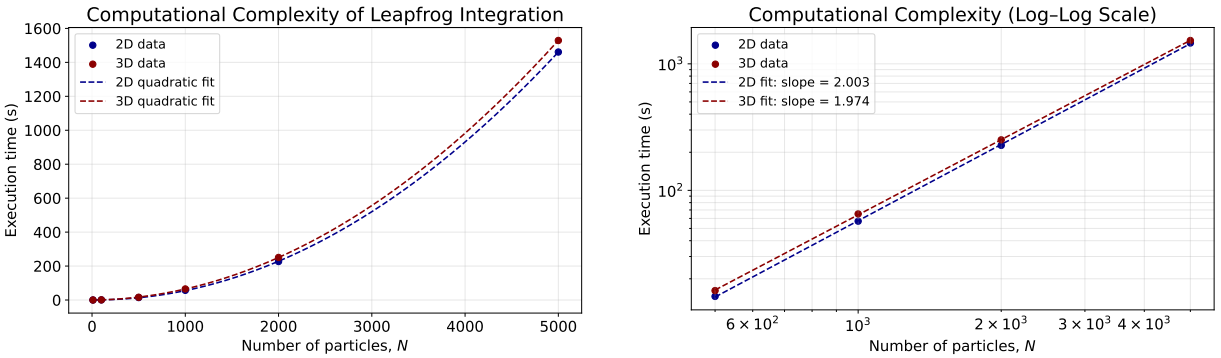
where $a$, $b$, and $c$ are coefficients determined by least-squares fitting in Python. As shown in Fig. 5(a), the quadratic functions closely follow the measured execution times for both dimensions, supporting the theoretically predicted $\mathcal{O}(N^2)$ scaling.
To independently estimate the computational scaling exponent, the slope $m$ of a linear fit in logarithmic space was found. The slope of this fit corresponds to the computational complexity exponent [8]. For this analysis, only measurements with $N \geq 500$ were included in the linear regression. At smaller particle counts, fixed computational overhead, including initialization and file operations, represents a larger fraction of the total execution time. These contributions can distort the measured scaling exponent. For larger systems, the gravitational force calculations dominate, making the measured scaling more representative of the computational complexity of the algorithm itself.
The resulting log–log fits, shown in Fig. 5(b), give slopes of

$$m_{\text{2D}} = 2.003, \qquad m_{\text{3D}} = 1.974. \tag{5.5}$$

Both values are close to the theoretical prediction of $m = 2$, confirming that the measured execution times exhibit approximately quadratic scaling with the number of particles.
The execution times are consistently greater in three dimensions due to the additional arithmetic operations required to evaluate each gravitational interaction. However, the scaling exponent remains approximately the same in both dimensions. These results demonstrate that the leapfrog implementation has the expected computational complexity of $\mathcal{O}(N^2)$.

## 6. Runge–Kutta Method

The Runge–Kutta method was implemented as an alternative numerical integration scheme to the leapfrog method, allowing a comparison of their accuracy and energy conservation properties. Runge–Kutta methods solve ordinary differential equations by evaluating the derivatives at multiple intermediate points within each time step. In this study, fourth-order (RK4) and eighth-order (RK8) methods were used to integrate the equations of motion of the $N$-body system. Their performance was compared with the leapfrog method by examining the evolution of total mechanical energy and its deviation from the initial value.




(a) Quadratic polynomial fits

(b) Log–log power-law fits

FIG. 5. Computational complexity of the optimized leapfrog integration scheme in two and three dimensions for 1000 integration steps. (a) Measured execution times and quadratic polynomial fits for $N = 10$ to 5000. (b) Log–log plots and power-law fits restricted to $N \geq 500$ to reduce the influence of fixed computational overhead. The fitted slopes of 2.003 and 1.974 agree closely with the theoretical $\mathcal{O}(N^2)$ scaling.

### 6.1. Fourth-Order Runge-Kutta Method

The RK4 method evaluates four intermediate derivatives at each timestep and then takes a weighted average. For the gravitational $N$-body problem, the equations of motion are second-order differential equations,

$$\frac{d^2\boldsymbol{x}_i}{dt^2} = \boldsymbol{a}_i(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N), \tag{6.1}$$

where the acceleration of each particle depends on the positions of all other particles.
To apply RK4, the second-order equation is rewritten as a system of two first-order differential equations by introducing the velocity $\boldsymbol{v}_i$:

$$\begin{aligned} \frac{d\boldsymbol{x}_i}{dt} &= \boldsymbol{v}_i, \\ \frac{d\boldsymbol{v}_i}{dt} &= \boldsymbol{a}_i(\boldsymbol{x}). \end{aligned} \tag{6.2}$$

Since the gravitational acceleration depends only on position, the equations can be integrated by calculating two sets of RK4 coefficients simultaneously. These coefficients are given in Table II.

TABLE II. Intermediate coefficients for the fourth-order Runge–Kutta integration of the gravitational $N$-body equations of motion.

| Stage | Position coefficient | Velocity coefficient |
|---|---|---|
| 1 | $\boldsymbol{k}_1 = \boldsymbol{v}_n$ | $\boldsymbol{l}_1 = \boldsymbol{a}(\boldsymbol{x}_n)$ |
| 2 | $\boldsymbol{k}_2 = \boldsymbol{v}_n + \frac{h}{2}\boldsymbol{l}_1$ | $\boldsymbol{l}_2 = \boldsymbol{a}\left(\boldsymbol{x}_n + \frac{h}{2}\boldsymbol{k}_1\right)$ |
| 3 | $\boldsymbol{k}_3 = \boldsymbol{v}_n + \frac{h}{2}\boldsymbol{l}_2$ | $\boldsymbol{l}_3 = \boldsymbol{a}\left(\boldsymbol{x}_n + \frac{h}{2}\boldsymbol{k}_2\right)$ |
| 4 | $\boldsymbol{k}_4 = \boldsymbol{v}_n + h\boldsymbol{l}_3$ | $\boldsymbol{l}_4 = \boldsymbol{a}\left(\boldsymbol{x}_n + h\boldsymbol{k}_3\right)$ |

Once the eight coefficients have been evaluated, the particle positions and velocities are updated according to

$$\begin{aligned} \boldsymbol{x}_{n+1} &= \boldsymbol{x}_n + \frac{h}{6}(\boldsymbol{k}_1 + 2\boldsymbol{k}_2 + 2\boldsymbol{k}_3 + \boldsymbol{k}_4), \\ \boldsymbol{v}_{n+1} &= \boldsymbol{v}_n + \frac{h}{6}(\boldsymbol{l}_1 + 2\boldsymbol{l}_2 + 2\boldsymbol{l}_3 + \boldsymbol{l}_4). \end{aligned} \tag{6.3}$$

### 6.2. Eighth-Order Runge–Kutta Method

The eighth-order Runge–Kutta (RK8) method follows the same principle as RK4 but uses additional intermediate stages to achieve higher numerical accuracy.
The DOP853-based fixed-step implementation uses twelve stages with weights $b_i$ and coefficients $a_{ij}$ from the Butcher tableau. The coefficient arrays were exported from SciPy [9, 10].

For the gravitational $N$-body problem, the second-order equations of motion are separated into the same two first-order equations for position and velocity used in RK4. The intermediate coefficients for position and velocity are again denoted by $\boldsymbol{k}_i$ and $\boldsymbol{l}_i$, respectively.
At each time step, these coefficients are calculated using

$$\begin{aligned} \boldsymbol{k}_i &= \boldsymbol{v}_n + h\sum_{j=1}^{i-1} a_{ij}\boldsymbol{l}_j, \\ \boldsymbol{l}_i &= \boldsymbol{a}\left(\boldsymbol{x}_n + h\sum_{j=1}^{i-1} a_{ij}\boldsymbol{k}_j\right). \end{aligned} \tag{6.4}$$

Once all 12 stages have been evaluated, the positions and velocities are updated using

$$\begin{aligned} \boldsymbol{x}_{n+1} &= \boldsymbol{x}_n + h\sum_{i=1}^{12} b_i\boldsymbol{k}_i, \\ \boldsymbol{v}_{n+1} &= \boldsymbol{v}_n + h\sum_{i=1}^{12} b_i\boldsymbol{l}_i. \end{aligned} \tag{6.5}$$

### 6.3. Trajectory Accuracy

The trajectory of $N = 100$ particles was simulated for all three integration schemes. Each simulation was performed over the interval $t = 100$ with a fixed time step $\Delta t = 0.001$ and gravitational softening parameter $\epsilon = 0.01$. Identical initial conditions were used for all three methods. Visually, the 3D trajectories appeared to be the same for all three integration shcemes. To quantitatively compare the numerical integration methods, the mean difference in particle positions was calculated between corresponding particles in each simulation. As shown in Fig. 6, the trajectories obtained using RK4 and RK8 remain nearly identical throughout the simulation. The differences between leapfrog and the Runge–Kutta methods increase gradually over time, reaching approximately $5.7 \times 10^{-6}$ at $t = 100$.
These results demonstrate that all three methods produce very similar trajectories for the chosen time step of $\Delta t = 0.001$. The close agreement between RK4 and RK8 suggests that the Runge–Kutta solutions are highly consistent, while the small deviations from leapfrog reflect differences in the numerical integration schemes.

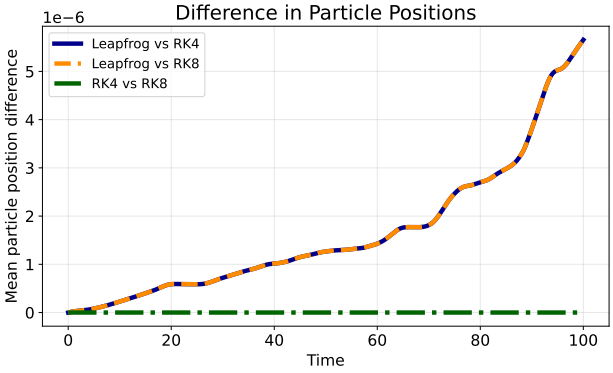



FIG. 6. Mean particle position differences between leapfrog, fourth-order Runge–Kutta (RK4), and eighth-order Runge–Kutta (RK8) integration methods for the 3D $N = 100$ gravitational system. Simulations were performed using identical initial conditions and a time step of $\Delta t = 0.001$ over a total simulation time of $t = 100$. The differences were calculated by averaging the Euclidean distance between corresponding particle positions at each time step.

### 6.4. Time Reversibility

The time reversibility of the three integration methods was investigated by evolving a system of $N = 100$ particles forward in time and then integrating backward using the negative timestep $-\Delta t$. The final state of the forward simulation was used as the initial state of the backward simulation. Numerical reversibility was evaluated by calculating the mean position recovery error of the particles throughout the entire backwards trajectory,

$$\delta r = \frac{1}{N}\sum_{i=1}^{N} \left\| \mathbf{r}_i^{\text{recovered}} - \mathbf{r}_i^{\text{initial}} \right\|. \tag{6.6}$$

Simulations were performed using $\Delta t = 0.001$ and a gravitational softening parameter of $\epsilon = 0.01$, with integration lengths ranging from 10 to 10,000 steps. The results are presented in Figure 7.

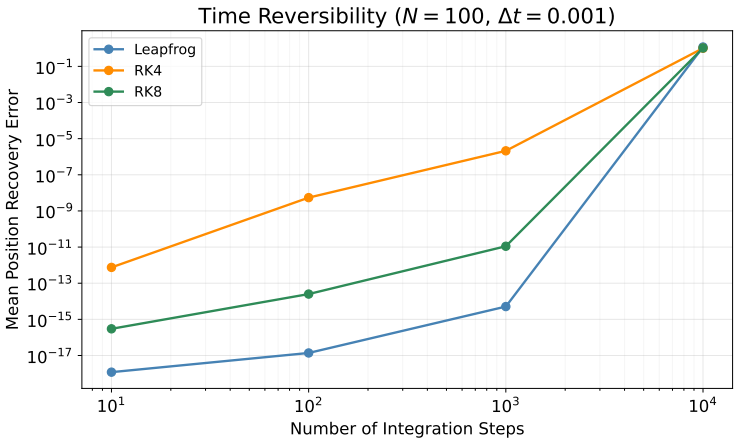



FIG. 7. Mean position recovery error as a function of the number of integration steps for leapfrog, fourth-order Runge–Kutta (RK4), and eighth-order Runge–Kutta (RK8), with $N = 100$, $\Delta t = 0.001$, and $\epsilon = 0.01$.

For short integration times, leapfrog exhibited the smallest recovery errors, remaining near machine precision for 10 and 100 steps and reaching approximately $5.1 \times 10^{-15}$ after 1,000 steps. This behavior is consistent with the time-symmetric formulation of the leapfrog method. In comparison, RK4 and RK8 exhibited larger recovery errors, as neither method is inherently time-reversible. At 1,000 steps, the mean position errors were approximately $2.1 \times 10^{-6}$ for RK4 and $1.1 \times 10^{-11}$ for RK8.
After 10,000 steps, all three methods exhibited substantial deviations from their initial configurations, with mean position recovery errors of approximately 1. This suggests that small numerical errors were strongly amplified over longer integration times.
To investigate the influence of gravitational softening, the leapfrog simulation was repeated for 10,000 steps with $\epsilon = 0.1$. Increasing the softening parameter reduced the mean position recovery error from approximately 1.18 to $1.64 \times 10^{-12}$. This substantial improvement is consistent with gravitational softening reducing the strength of close encounters and limiting the amplification of numerical errors. However, increasing $\epsilon$ also modifies the gravitational interactions at short distances and therefore changes the physical system being simulated.
These results demonstrate that numerical time reversibility depends not only on the integration scheme but also on the system parameters. Although leapfrog is time-symmetric, finite-precision errors can still lead to substantial trajectory divergence over sufficiently long simulations.

### 6.5. Conservation Properties

The conservation properties of the leapfrog, fourth-order Runge–Kutta (RK4), and eighth-order Runge–Kutta (RK8) integration methods were compared for the same three-dimensional system of $N = 100$ particles.

#### 6.5.1. Energy Conservation.

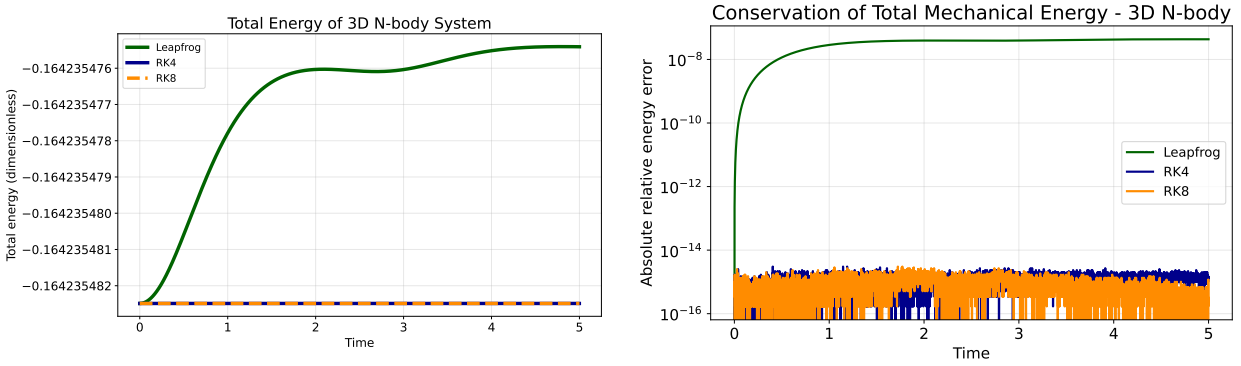
Figure 8 compares the evolution of the total energy and its relative deviation from the initial value. The RK4 and RK8 methods exhibit nearly constant total energy, with relative deviations on the order of $10^{-15}$, approaching machine precision. In comparison, the leapfrog method exhibits a larger relative energy deviation at the start of the simulation, reaching approximately $10^{-8}$. However, the leapfrog energy error is still relatively small and remains constant after the initial increase. This bounded error does not by itself imply an inaccurate trajectory.

#### 6.5.2. Angular Momentum Conservation.

Analyzing the same simulations showed the angular momentum during each simulation and the relative deviation from the initial angular momentum vector. All three integration methods conserve angular momentum to approximately machine precision, with relative errors of order $10^{-15}$ or smaller. No significant systematic drift is observed over the simulation interval.

#### 6.5.3. Conservation of Barycentric Velocity.

The barycentric velocity was also monitored as an additional measure of numerical conservation. the corresponding diagnostic shows that its magnitude remains on the order of $10^{-16}$ for all three methods. The extremely small deviations confirm that the total linear momentum is conserved to approximately machine precision.



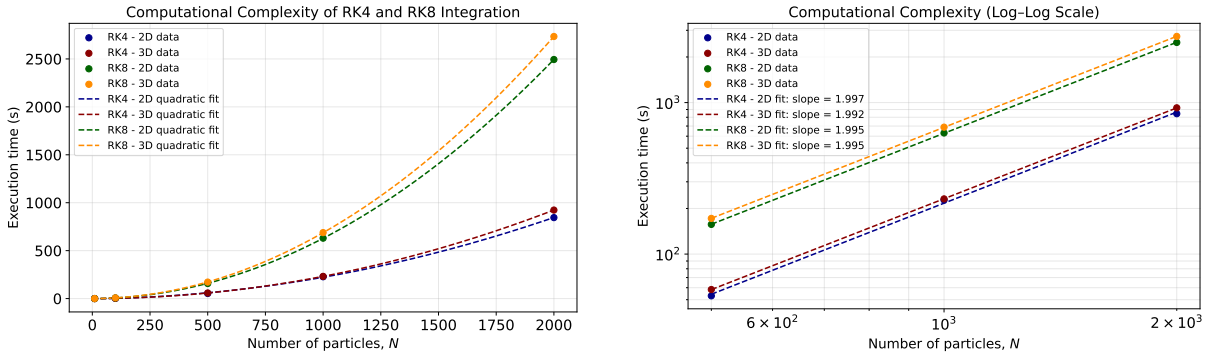

(a) Total mechanical energy

(b) Absolute relative energy deviation

FIG. 8. Comparison of energy conservation for the 3D $N = 100$ particle system using leapfrog, RK4, and RK8 integration methods. Panel (a) shows the evolution of total mechanical energy, while panel (b) shows the absolute relative energy deviation from its initial value.

### 6.6. Computational Efficiency

The computational complexity of the leapfrog method was found to be $\mathcal{O}(N^2)$ with $KN(N-1)$ pairwise interactions at each time step. For the RK4 implementation, the acceleration of each particle is calculated four times per time step, corresponding to $4KN(N-1)$ pairwise interactions. Therefore, the complexity of the RK4 method is expected to be $\mathcal{O}(N^2)$, but requires four times as many force calculations per particle. The RK8 method requires 12 acceleration evaluations per time step. Therefore, it is even more computationally expensive, but is still expected to scale quadratically with particle number.
Figure 9 confirms that the computational cost of both RK4 and RK8 do scale approximately as $O(N^2)$, with fitted log–log slopes between 1.992 and 1.997. RK8 requires approximately three times more execution time than RK4, while three-dimensional simulations are slightly more expensive than two-dimensional simulations.




(a) Quadratic least-squares fit.

(b) Log–log fit for $N \geq 500$.

FIG. 9. Computational complexity of RK4 and RK8 integration in two and three dimensions. The left panel shows quadratic least-squares fits to the measured execution times, while the right panel shows power-law fits on logarithmic axes. The slopes of the log–log fits estimate the computational scaling exponent $m$ in $T(N) \propto N^m$.

## 7. Discussion

### 7.1. Numerical Stability and Accuracy

The main source of error and discrepancy between numerical results were the parameters used during integration. These effects of varying $\epsilon$ and $\Delta t$ were investigated in detail for the leapfrog method, revealing the effect of these paremeters on the predicted motion of the particles. These discrepancies are likely due to the large gravitational forces that occur between particles during close interactions. When the time step, is too large, it cannot account for these large increases and decreases in force that occur relatively fast. Introducing gravitational softening reduced the magnitude of these forces during close encounters, and improved the stability of the system. However, increasing the softening parameter also modified the gravitational force, reducing the accuracy of the calculations and therefore the dynamics of the system.
Therefore, the value $\epsilon = 0.01$ was selected as a reasonable compromise between numerical stability and preserving the gravitational interactions. Similarly, reducing the time step to $\Delta t = 0.001$ substantially improved energy conservation compared with the larger time steps investigated. However, the observed

differences between particle trajectories when parameters are varied indicate that more detailed studies would be needed to select the ideal parameter values.
The conservation of energy, angular momentum and barycentric velocity provided additional validation of the leapfrog integration method. The total mechanical energy was conserved when the suitable values of $\epsilon$ and $\Delta t$ discussed above were chosen. Angular momentum and barymetric velocity were also conserved, to almost machine precision. This result demonstrated their indifference to the time step and softening parameters and validated the integration method for predicting the dynamics of the system.

### 7.2. Accuracy Versus Computational Cost

The comparison of leapfrog, RK4, and RK8 demonstrated that all three integration schemes produced very similar particle trajectories for the selected simulation parameters. The mean difference between leapfrog and the Runge–Kutta solutions remained small, reaching approximately $5.7 \times 10^{-6}$ at $t = 100$, while RK4 and RK8 produced nearly identical trajectories.
The conservation analysis revealed differences between the integration methods. RK4 and RK8 maintained total mechanical energy with relative deviations approaching machine precision, while leapfrog exhibited larger energy fluctuations. These results suggest that the higher-order Runge–Kutta methods provide greater energy accuracy for the particular time step and simulation interval investigated.
The computational cost must also be considered when selecting an integration scheme. The optimized leapfrog method requires approximately one gravitational acceleration evaluation per time step, compared with four for RK4 and twelve for RK8. Since each acceleration evaluation requires $\mathcal{O}(N^2)$ operations, the additional stages increase the computational expense substantially, particularly for large systems.

### 7.3. Limitations of the Model

Several simplifications were made in the physical model. All particles has identical masses and interacted exclusively through Newtonian gravity. The initial positions were generated randomly within a circle or sphere, while the initial velocities followed solid-body rotation. These conditions provide a convenient starting point for investigating gravitational dynamics but do not represent precisely a realistic astrophysical system.
The use of gravitational softening also introduces an approximation to the predicted dynamics of the system. While it was necessary to improve numerical stability, the addition of the softening parameter alters the interaction of particles during close encounters. The appropriate softening parameter therefore must be selected by compromising between limiting numerical errors and accurately representing the dynamics of the system.
Another limitation of the model is the computational cost. For all three integrations schemes, the computational complexity scaled quadratically with the number of particles. For complex system, such as a galaxy containing thousands of particles or bodies, this would result in high computational cost and long simulation times.
Finally, although conservation of energy and momentum were demonstrated and the trajectories of the particles agreed between models, this does not indicate accuracy of the trajectories. The error in backwards trajectory with the leapfrog integrator especially demonstrate that small errors in trajectories accumulate over time. To confirm the numerical accuracy of the integration methods, comparison with higher-accuracy reference solutions would be necessary.

## 8. Conclusion

In this project, the gravitational $N$-body problem was simulated in two and three dimensions using numerical integration methods implemented in Fortran. The leapfrog integration scheme was validated against the analytical solution for a two-body system and then applied to more complex 3D systems.
The investigation demonstrated that gravitational softening and integration time step have significant effects on the numerical stability and evolution of the system. A softening parameter of $\epsilon = 0.01$ and a time step of $\Delta t = 0.001$ were selected as reasonable parameters for the simulations considered. Angular momentum and linear momentum were conserved to approximately machine precision, while energy conservation was found to be more sensitive to the numerical parameters. Reversibility of all integration methods was shown to hold only over small time scales, with the leapfrog demonstrating overall better reversibility due to its symplectic nature.
The computational complexity of the $N$-body method was confirmed to scale approximately as $\mathcal{O}(N^2)$ for all three integration methods. The comparison of leapfrog, RK4, and RK8 showed close agreement between the predicted trajectories, with the Runge–Kutta methods producing smaller energy deviations for the conditions investigated.
Overall, the results highlight the importance of balancing numerical stability, physical accuracy, and computational efficiency when selecting an integration method. While higher-order methods can provide greater accuracy for a given time step, the lower computational cost and symplectic properties of leapfrog make it a useful approach for gravitational $N$-body simulations. Further investigations could focus on numerical convergence, long-term system behaviour, and more efficient gravitational force calculation methods for larger systems.

# Appendix A: Functional Requirements of the Program

1. **Purpose:** Simulate the gravitational motion of an $N$-body system using Leapfrog, RK4, or RK numerical integration.

2. **Inputs:** User selection of simulation type, integration type and integration direction. User can also modify the simulation parameters shown in Table III. The coefficients stored in `rk8_coefficients.py` are also used for the RK8 integration method.

TABLE III. User-configurable simulation parameters.

| Parameter | Allowed values | Description |
|---|---|---|
| `simulation_name` | String | Simulation identifier |
| `N` | Integer, $N \geq 2$ | Number of particles |
| `dim` | 2 or 3 | Spatial dimensions |
| `iterations` | Positive integer | Number of time steps |
| `dt` | Real, $dt > 0$ | Time step size |
| `eps` | Real, $\epsilon \geq 0$ | Softening parameter |
| `omega` | Real | Initial angular velocity |
| `output_filename` | String | Particle trajectory file |
| `quantity_filename` | String | Conservation data file |
| `output_backwards_filename` | String | Reverse trajectory file |
| `quantity_backwards_filename` | String | Reverse conservation file |

3. **Outputs:** Text files containing particle positions and velocities at each time step, along with total energy and angular momentum for conservation analysis. User can specify the text file names in the main program as shown in Table III.

4. **Post-processing:** Python scripts using NumPy and Matplotlib visualize particle trajectories, evaluate conservation errors, and compare the accuracy and computational cost of the integration methods.

5. **Requirements:** A Fortran compiler (e.g., GNU Fortran). The post processing script requires Python with NumPy, SciPy, and Matplotlib. The RK8 implementation additionally requires a file containing the DOP853 coefficients.

# Appendix B: Internal program structure

The main program `main.f90` reads the user's simulation and integration choices, sets the parameters, and allocates the necessary arrays. The `initialize` module generates the initial particle positions and velocities and removes the barycentric velocity. The selected integration method (Leapfrog, RK4, or RK8) in `integrate` then evolves the system using gravitational accelerations calculated by the `forces` module, which also computes conserved quantities. If requested, the simulation is repeated backward in time. The program measures the total execution time, while RK8 additionally uses coefficients generated by a Python script.

**Modules and Subroutines**

### a. Initialization Module

The `initialize` module generates the initial positions and velocities of the particles using rejection sampling and solid-body rotation. It also calculates the barycentric velocity.

TABLE IV. Inputs and outputs of the initialization subroutines.

| Subroutine | Inputs | Output |
|---|---|---|
| `get_position_components` | dim, R | position(dim) |
| `get_velocity_components` | position, dim, omega | velocity(dim) |
| `barycentric_velocity` | vel, N, dim | v_bar(dim) |

### b. Forces Module

The `forces` module calculates gravitational forces, particle accelerations, and the physical quantities used to evaluate conservation.

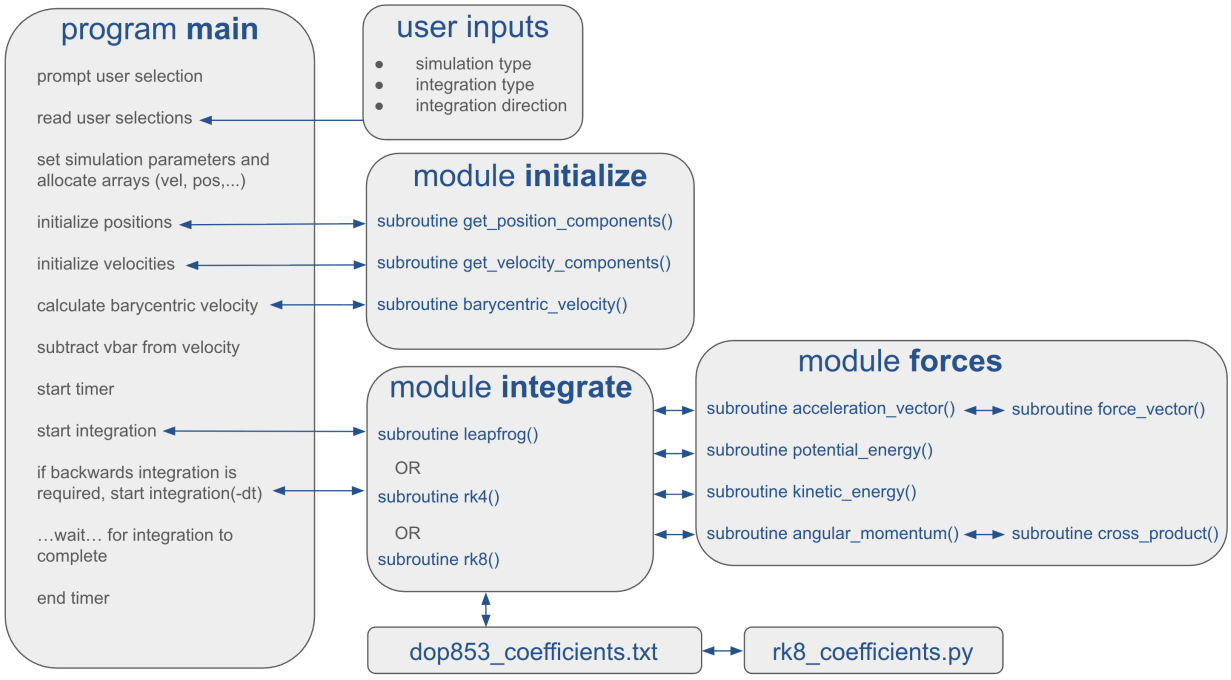



FIG. 10. Functional diagram of the N-body simulation program. Arrows indicate communication between the program and its modules.

TABLE V. Inputs and outputs of the forces subroutines.

| **Subroutine** | **Inputs** | **Output** |
|---|---|---|
| `force_vector` | pos_i, pos_j, N, dim, eps | F(dim) |
| `acceleration_vector` | pos, N, dim, i, mass, eps | Acc(dim) |
| `potential_energy` | pos, N, dim, eps | Epot |
| `kinetic_energy` | vel, N, dim | Ekin |
| `angular_momentum` | pos, vel, N, dim | L(3) |
| `cross_product` | a, b, dim | c(3) |

### c. Integration Module

The `integrate` module implements three numerical integration methods: leapfrog, fourth-order Runge–Kutta (RK4), and eighth-order Runge–Kutta (RK8). All three methods use the same input and output variables.

TABLE VI. Inputs and outputs of the integration subroutines.

| **Variable** | **Direction** | **Description** |
|---|---|---|
| `pos(N,dim)` | In/Out | Particle positions |
| `vel(N,dim)` | In/Out | Particle velocities |
| `dt` | Input | Time step |
| `N` | Input | Number of particles |
| `dim` | Input | Spatial dimensions |
| `iterations` | Input | Number of steps |
| `mass` | Input | Particle mass |
| `eps` | Input | Softening parameter |
| `save_filename` | Input | Trajectory output file |
| `quantity_filename` | Input | Conservation output file |

The integration subroutines update the particle positions and velocities directly and save the results at each time step. Separate output files contain the particle trajectories and conserved quantities.

The leapfrog method uses the Kick–Drift–Kick formulation and reuses previously calculated accelerations. RK4 uses four intermediate stages, while RK8 uses twelve stages with coefficients read from `dop853_coefficients.txt`. These coefficients are generated using the Python script `rk8_coefficients.py`, which extracts the DOP853 coefficients from SciPy.

**Routine Example - Leapfrog Method**

## integrate.leapfrog

**leapfrog**(pos, vel, dt, N, dim, iterations, mass, eps, save_filename, quantity_filename)

Update position and velocity over time using leapfrog kick-drift-kick scheme.

**Parameters:**

pos(N,dim): real array (In/Out)
Particle positions.

vel(N,dim): real array (In/Out)
Particle velocities.

dt: real (Input)
Integration time step.

N: integer (Input)
Number of particles.

dim: integer (Input)
Number of spatial dimensions (2 or 3).

iterations: integer (Input)
Number of integration steps.

mass: real (Input)
Mass of each particle.

eps: real (Input)
Gravitational softening length.

save_filename: character string (Input)
File path for particle positions and velocities.

quantity_filename: character string (Input)
File path for conserved quantities.

FIG. 11. Overview of the `leapfrog` subroutine and its parameters.

# Appendix C: Quality approach and reliability

A thorough description of the ways the numerical accuracy and stability of the program were tested is outlined in the main report. The following points summarize this procedure and highlight the effort made to ensure the program did not just run, but produced satisfactory results.

- **Analytical check:** compare the two-body circular orbit and its period with the analytical prediction.
- **Reproducibility:** use a fixed random seed when varying $\epsilon$, $\Delta t$, or integrator.
- **Conservation checks:** track total energy, angular momentum, and barycentric velocity to verify numerical stability.
- **Parameter sensitivity:** compare multiple softening lengths and time steps. Use these results to determine suitable parameters to be used for subsequent testing.
- **Independent methods:** compare leapfrog, RK4, and RK8 positions and conserved quantities under identical initial conditions.
- **Reversibility:** integrate forward and backward and measure the recovered-position error.
- **Complexity:** compare measured runtime scaling against the theoretical $\mathcal{O}(N^2)$ complexity predictions.

# Appendix D: Additional Plots

The following supplementary plots support the main discussion.

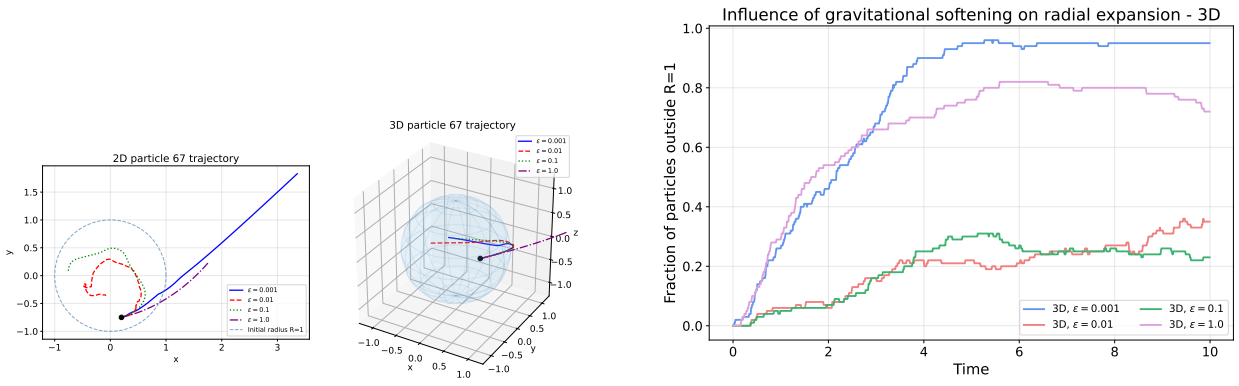



FIG. 12. Additional gravitational-softening tests: particle trajectories (left) and radial expansion (right).

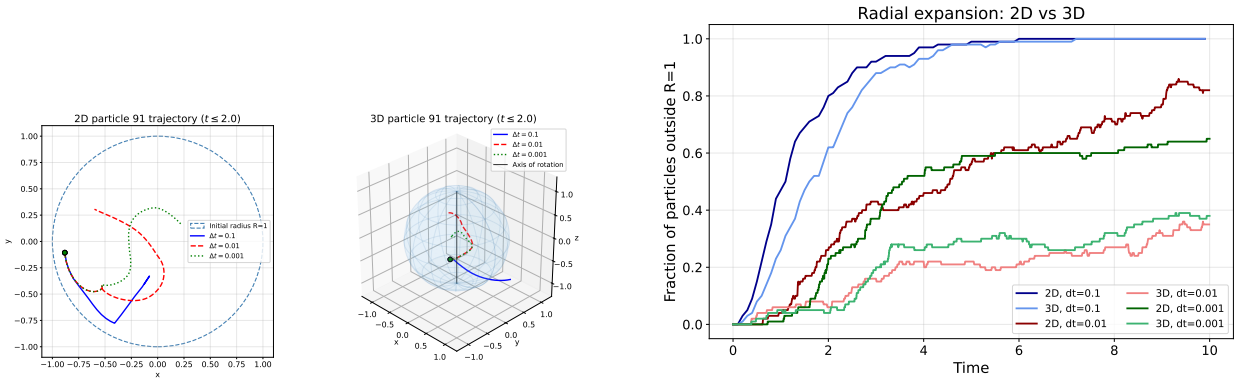



FIG. 13. Additional timestep comparisons: individual trajectories (left) and radial expansion (right).

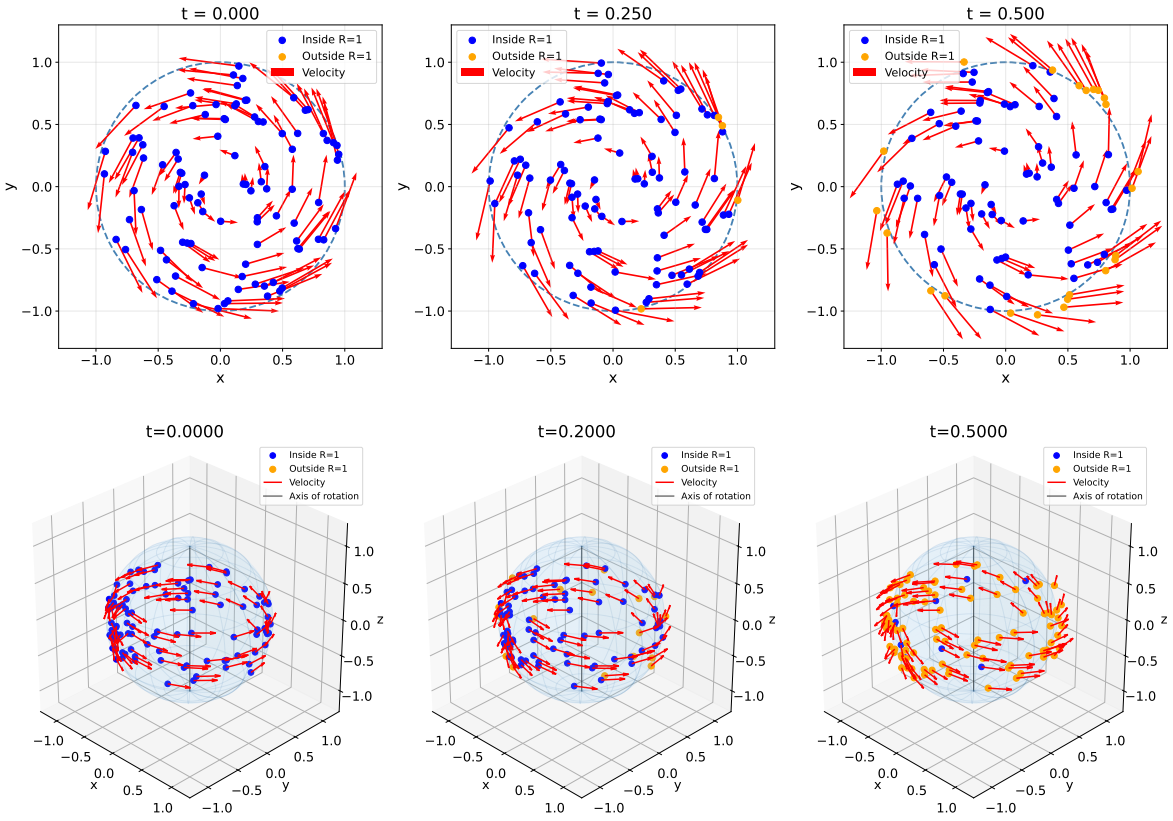
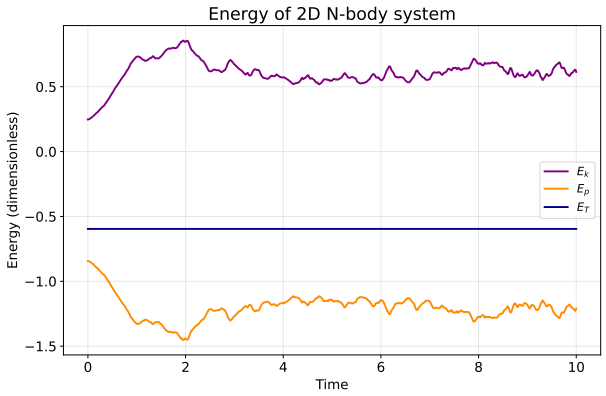
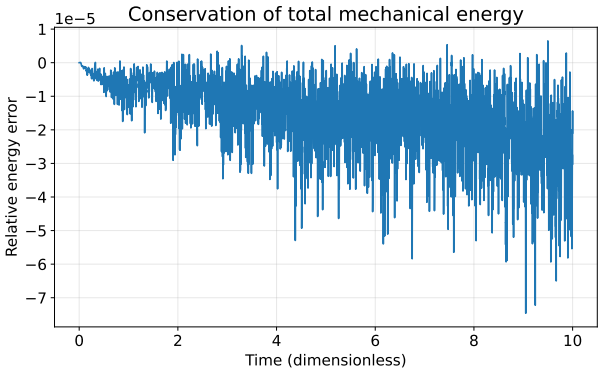



FIG. 14. Visualization of the evolution of the $N = 1000$ particle system in two and three dimensions at $t = 0$, $t = 0.25$, and $t = 0.5$. The upper panels show the 2D simulation, with 100 particles displayed, while the lower panels show the 3D simulation. All 1000 particles are included in the gravitational calculations. Blue particles remain within the initial radius $R = 1$, while orange particles have moved beyond it. Red arrows indicate particle velocities, and dashed circles or translucent spheres represent the initial spatial boundary.

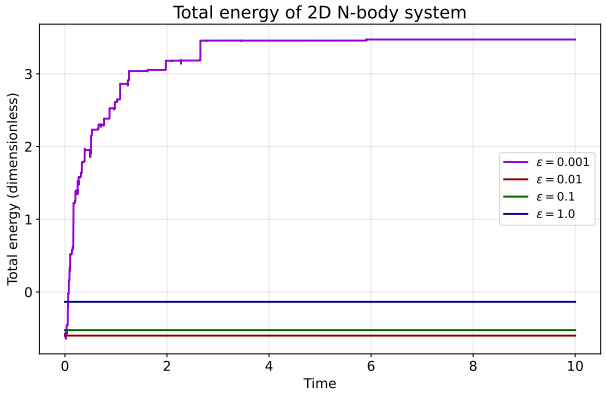



(a) Kinetic, potential and total mechanical energy.

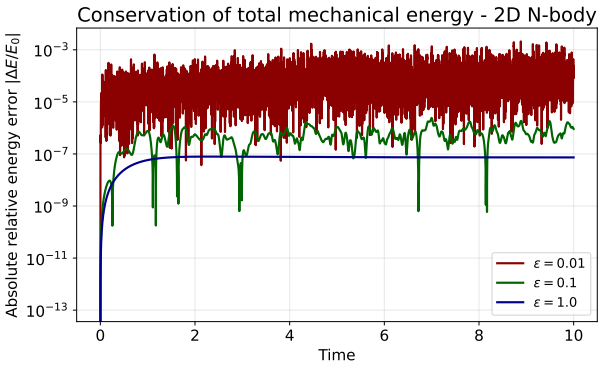



(b) Relative energy deviation.

FIG. 15. Energy evolution and conservation in the 2D 1000-body simulation. (a) Kinetic, potential, and total mechanical energy as functions of time. (b) Relative deviation of total energy from its initial value. The total energy remains approximately constant, with a relative error of order $10^{-6}$, demonstrating good energy conservation throughout the simulation.

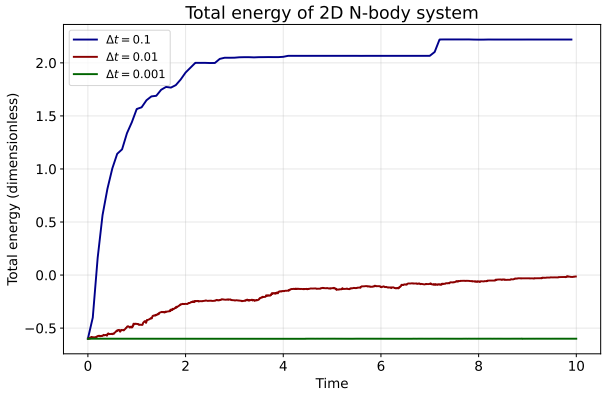



(a) Total mechanical energy

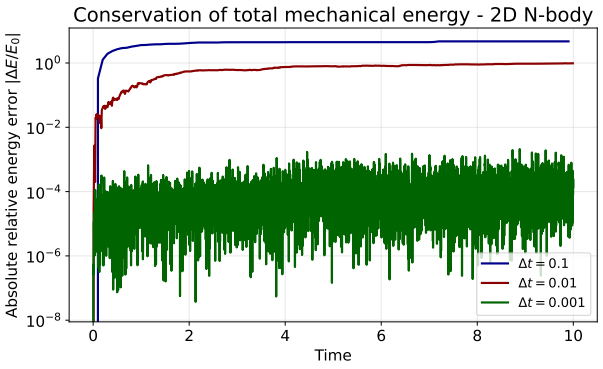



(b) Absolute relative energy deviation

FIG. 16. Comparison of total mechanical energy conservation for the 2D $N = 100$ particle system over $t = 10$, using a time step of $\Delta t = 0.001$ and different gravitational softening parameters. Panel (a) shows the evolution of total mechanical energy for $\epsilon = 0.001$, 0.01, 0.1, and 1.0. Panel (b) shows the corresponding absolute relative energy deviation for $\epsilon = 0.01$, 0.1, and 1.0 on a logarithmic scale. The $\epsilon = 0.001$ case is excluded from panel (b) due to its substantially larger energy error. Increasing the softening parameter reduces the energy deviations, with $\epsilon = 1.0$ exhibiting the smallest variation over the simulated interval.

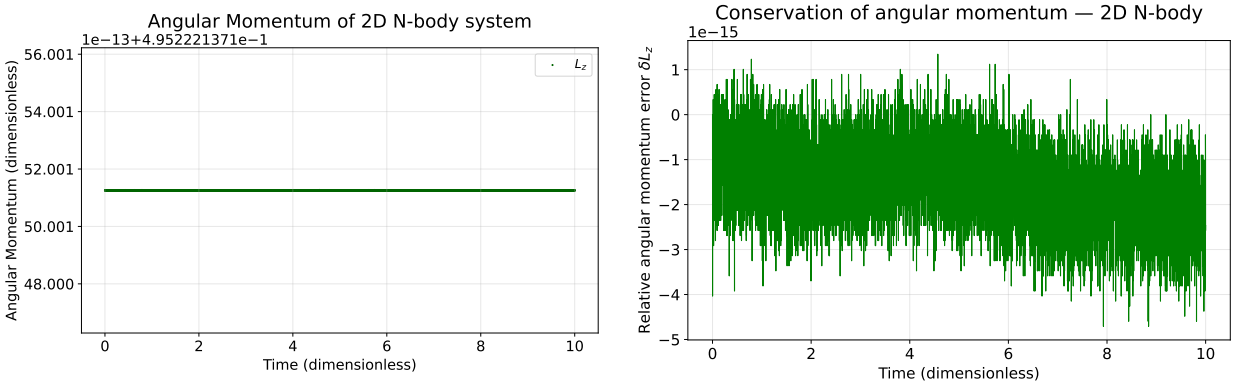



(a) Total mechanical energy




(b) Absolute relative energy deviation

FIG. 17. Energy conservation in the 2D $N = 100$ particle system for $\Delta t = 0.1$, 0.01, and 0.001, with gravitational softening $\epsilon = 0.01$. Panel (a) shows the total mechanical energy, while panel (b) shows its absolute relative deviation on a logarithmic scale.




(a) Total angular momentum $L_z$

(b) Relative angular momentum error

FIG. 18. Conservation of angular momentum in the 2D $N = 100$ particle system over $t = 10$. Panel (a) shows the evolution of the total angular momentum $L_z$, while panel (b) shows its relative deviation from the initial value, $\delta L_z = [L_z(t) - L_z(0)]/|L_z(0)|$. The relative deviations remain on the order of $10^{-15}$, demonstrating conservation of angular momentum to approximately machine precision.

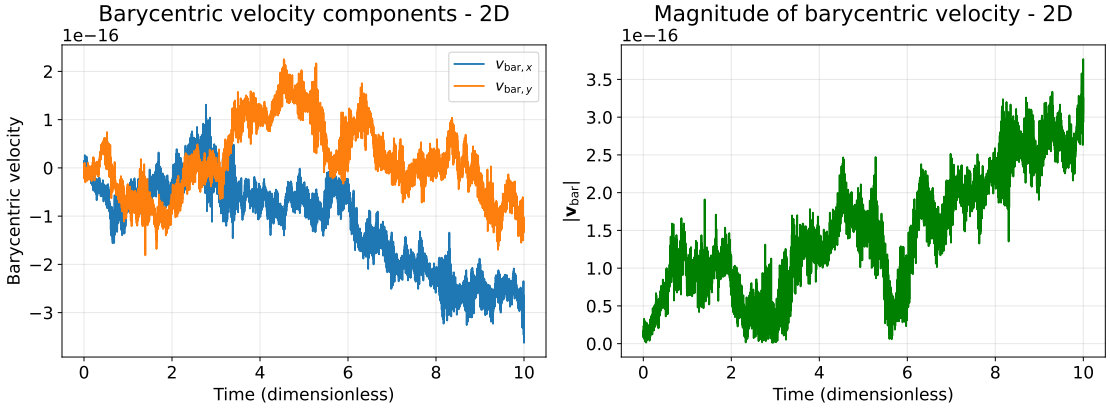
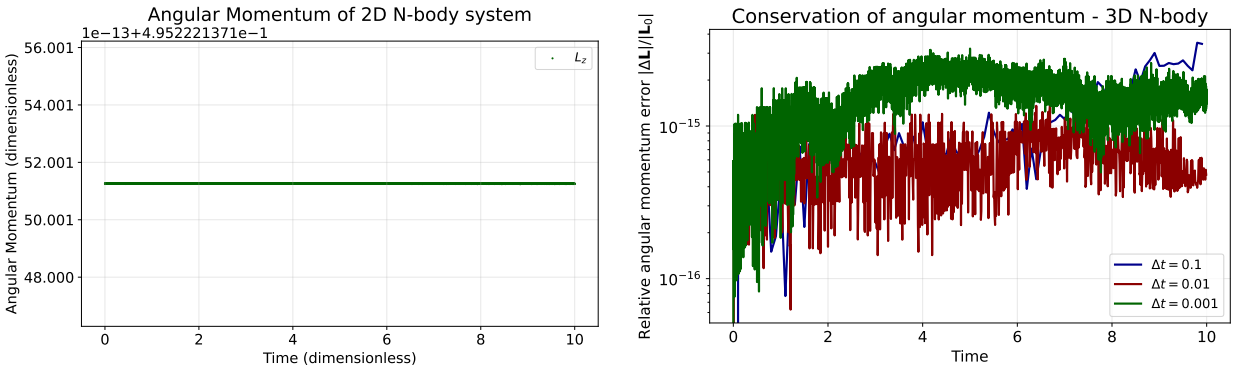



FIG. 19. Conservation of barycentric velocity in the 2D $N = 100$ particle system over $t = 10$. The left panel shows the evolution of the barycentric velocity components, $v_{\mathrm{bar},x}$ and $v_{\mathrm{bar},y}$, while the right panel shows the magnitude $|\boldsymbol{v}_{\mathrm{bar}}|$. The barycentric velocity remains on the order of $10^{-16}$, demonstrating conservation of linear momentum to approximately machine precision.




(a) Angular momentum components

(b) Relative angular momentum deviation

FIG. 20. Angular momentum conservation in the 3D $N = 100$ particle system for $\Delta t = 0.1$, 0.01, and 0.001. Panel (a) shows the three angular momentum components, while panel (b) shows the relative deviation on a logarithmic scale.

[1] Spiteri, course material on the gravitational $N$-body problem (bibliographic details to confirm).
[2] Wikipedia contributors, "Three-body problem," https://en.wikipedia.org/wiki/Three-body_problem.
[3] Reference on barycentric motion (original citation details to confirm).
[4] University of California, San Diego, "Leapfrog Integration," Physics 141 (2019), https://courses.physics.ucsd.edu/2019/Winter/physics141/Assignments/leapfrog.pdf.
[5] Bohrium, "Plummer Potential," https://www.bohrium.com/en/sciencepedia/feynman/keyword/plummer_potential.
[6] Wikipedia contributors, "N-body problem," https://en.wikipedia.org/wiki/N-body_problem.
[7] Wikipedia contributors, "Momentum," https://en.wikipedia.org/wiki/Momentum.
[8] Computational complexity course notes (bibliographic details to confirm).
[9] Lecoutre, "TP: Function Zeros," Numerical Methods Practical, Université Marie et Louis Pasteur (2026).
[10] SciPy Developers, "scipy.integrate.DOP853," https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.DOP853.html.